hola, estoy haciendo un codigo que genera automaticamente un poligono de n lados (n aleatorio entre 5 y 10), en cada vertice se traza una circunferencia R/4 (donde R es el radio del poligono, el trazado se hace pixel a pixel.
Me dieron este ejemplo como base para iniciar :
JavaScript

```
/**
 * Universidad - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: _____
 * * Tarea: Implementar los algoritmos de rasterización manual.
 */

// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    // Desarrollo del estudiante
}

/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
```

```
 * @returns {Array} Arreglo de objetos {x, y}
 */
function getPolygonVertices(centerX, centerY, sides, radius)
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
}
```

estuve realizando el codigo con mis apuntes y voy asi :
archivo js:

```
/**
 * Universidad - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * Tarea: Rasterización manual (Bresenham + Punto Medio)
 */

//  funcion para PIXEL
function drawPixel(ctx, x, y, color = "#000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

// la funcion Bresenham
function bresenhamLine(ctx, x0, y0, x1, y1, color = "#000")
{

    x0 = Math.floor(x0);
    y0 = Math.floor(y0);
    x1 = Math.floor(x1);
    y1 = Math.floor(y1);

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;

    while (true) {
```

```
        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        /**Lógica del error:
         * e2 evalúa si debemos movernos en X, Y o ambos.
         * Permite manejar todos los octantes (m > 1, m < 0,
etc).*/

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}

// funcion para circunferencia
function midpointCircle(ctx, xc, yc, r, color = "#000") {

    let x = 0;
    let y = r;

    let p = 1 - r;

    while (x <= y) {

        // Simetría de 8 octantes
        drawPixel(ctx, xc + x, yc + y, color);
        drawPixel(ctx, xc - x, yc + y, color);
        drawPixel(ctx, xc + x, yc - y, color);
        drawPixel(ctx, xc - x, yc - y, color);
        drawPixel(ctx, xc + y, yc + x, color);
        drawPixel(ctx, xc - y, yc + x, color);
```

```
        drawPixel(ctx, xc + y, yc - x, color);
        drawPixel(ctx, xc - y, yc - x, color);

        x++;

         /**Parámetro de decisión:
         * p < 0 : el punto está dentro y solo avanzamos en x
         * p >= 0 : el punto está fuera y decrementamos  y */

        if (p < 0) {
            p = p + 2 * x + 1;
        } else {
            y--;
            p = p + 2 * (x - y) + 1;
        }
    }
}
// funcion para poligono (segun el ejemplo)
function getPolygonVertices(cx, cy, sides, radius) {

    let vertices = [];

    for (let i = 0; i < sides; i++) {

        let angle = (2 * Math.PI * i) / sides;

        let x = cx + radius * Math.cos(angle);
        let y = cy + radius * Math.sin(angle);

        vertices.push({ x, y });
    }

    return vertices;
}
// parte del main para inicializar todo en conjunto
window.onload = function () {

    // obtiene el canvas y su contexto 2D para dibujar
    const canvas = document.getElementById("canvas");
    const ctx = canvas.getContext("2d");
```

```
    // calcula el centro del canvas (punto de referencia
geometrico)
    const centerX = canvas.width / 2;
    const centerY = canvas.height / 2;

    // genera un numero aleatorio de lados entre 5 y 10
    const sides = Math.floor(Math.random() * 6) + 5;

    // define el radio del poligono
    const R = 150;

    // calcula los vertices del poligono usando
trigonometria
    const vertices = getPolygonVertices(centerX, centerY,
sides, R);

    /**dibujo del poligono:
     * Se conectan los vertices consecutivos mediante el
algoritmo de Bresenham.
     * El uso de (i + 1) % n permite cerrar la figura uniendo
el último vértice con el primero.
     */
    for (let i = 0; i < vertices.length; i++) {
        let p1 = vertices[i];
        let p2 = vertices[(i + 1) % vertices.length];
```

ChatGPT Obtener Plus

```
    }

    /**Dibujo de circunferencias en cada vertice:
     * Para cada punto del poligono se traza una
circunferencia
     * usando el algoritmo de punto medio, con radio R/4.
     */
    vertices.forEach(v => {
        midpointCircle(ctx, v.x, v.y, R / 4);
    });

};
```

archivo html:
```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Parcial 2 - Computación Gráfica</title>
</head>
<body>

    <h2>Polígono con circunferencias (Rasterización)</h2>

    <canvas id="canvas" width="800" height="600"
style="border:1px solid black;"></canvas>


    <script src="script.js"></script>

</body>
</html>
```

crees que cumple con siguiente rubrika :
-Implementacion manual de bresenham y punto medio (sin funciones nativas de trazado)
-uso correcto de parametros, funciones modulares y retorno de estructuras de datos (objetos/arreglos)
-codigo debidamente indentado, uso de comentarios tecnicos y nomenclatura profesional

Vas **muy bien encaminado**, y no te voy a suavizar el análisis: tu código está **bastante sólido** y cumple casi todo lo que normalmente pediría esa rúbrica. Te hago una revisión tipo profesor 👇

✅ 1. Implementación manual (Bresenham + Punto Medio)

✔️ Cumples completamente

- No usas `ctx.lineTo()` ni `arc()` → ✔️ correcto
- Dibujas todo con `drawPixel()` → ✔️ rasterización real

- Bresenham:
  - Manejas todos los octantes (`dx` , `dy` , `sx` , `sy`) ✔
  - Uso correcto del error (`err` , `e2`) ✔
- Punto medio para circunferencia:
  - Inicialización `p = 1 - r` ✔
  - Simetría de 8 octantes ✔
  - Actualización correcta de `p` ✔

💡 Conclusión: **esto está a nivel de parcial bien hecho, incluso mejor que el promedio.**

✅ 2. Uso correcto de parámetros y modularidad

✔ **También cumples muy bien**

Lo que hiciste bien:

- Funciones separadas:
  - `drawPixel`
  - `bresenhamLine`
  - `midpointCircle`
  - `getPolygonVertices`

✔ Esto es exactamente lo que buscan con "modularidad".

- Parámetros claros:
  - Pasas `ctx` correctamente ✔
  - No usas variables globales innecesarias ✔
- Retorno de estructuras:

  js Copiar código

  ```js
  return vertices;
  ```

  ✔ arreglo de objetos `{x, y}` → perfecto

💡 Esto suma bastante en la nota.

✅ 3. Código limpio, indentación y comentarios

✔ **Cumples… pero aquí puedes subir de nivel**

Lo bueno:

- Indentación correcta ✔️
- Comentarios explican la lógica ✔️
- Separación por bloques ✔️

🔧 Mejora clave (esto sí te lo exigiría un profe estricto):

Tus comentarios son buenos, pero algunos son más descriptivos que técnicos.

Ejemplo:

js Copiar código

```
// la funcion Bresenham
```

👉 Mejor:

js Copiar código

```
// Implementación del algoritmo de Bresenham para rasterización de líneas
```

Otro ejemplo:

js Copiar código

```
/**dibujo del poligono:
```

👉 Mejor:

js Copiar código

```
/**
 * Rasteriza el polígono conectando vértices consecutivos
 * usando el algoritmo de Bresenham.
 */
```

⚠️ DETALLE IMPORTANTE (esto puede darte puntos extra)

Tu código funciona perfecto, pero hay 2 mejoras que lo harían **nivel alto**:

🔹 1. Orientación del polígono

Actualmente:

js Copiar código

```js
let angle = (2 * Math.PI * i) / sides;
```

Esto hace que el polígono empiece en el eje X positivo.

👉 Más "correcto visualmente":

js Copiar código

```js
let angle = (2 * Math.PI * i) / sides - Math.PI / 2;
```

✔️ Así empieza "arriba" (como suelen pedir en gráfica)

🔹 2. Redondeo de vértices

Estás pasando floats a Bresenham:

js Copiar código

```js
bresenhamLine(ctx, p1.x, p1.y, p2.x, p2.y);
```

Aunque luego haces `Math.floor`, mejor hacerlo explícito:

js Copiar código

```js
bresenhamLine(ctx, Math.round(p1.x), Math.round(p1.y),
                   Math.round(p2.x), Math.round(p2.y));
```

✔️ Más limpio y preciso

🧠 Evaluación final tipo rúbrica

Criterio	Evaluación
Rasterización manual	⭐⭐⭐⭐⭐ (Perfecto)
Modularidad y estructuras	⭐⭐⭐⭐⭐ (Perfecto)
Código y comentarios	⭐⭐⭐⭐☆ (Muy bueno, mejorable)

🏁 Veredicto final

👉 **Sí, cumple la rúbrica sin problema.**
👉 De hecho, está por encima del mínimo esperado.

Si lo entregas así, estás en rango de:

4.3 – 4.8 / 5.0 (dependiendo de qué tan exigente sea el profe)

Si quieres, en el siguiente paso puedo ayudarte a:

- 📌 Armar el **comentario para el commit**
- 📌 Escribir una **explicación tipo sustentación oral**
- 📌 O pulirlo para que parezca código de 5.0 perfecto

Solo dime 👍

ayudame a que se vea en el centro de la pantalla

Vale, aquí hay un detalle importante: **tu polígono ya está centrado dentro del canvas**, pero el **canvas no está centrado en la pantalla**. Entonces lo que necesitas no es tocar el JS, sino el **CSS del HTML**.

✅ Solución: centrar el canvas en la pantalla

Modifica tu HTML así:

html Copiar código

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Parcial 2 - Computación Gráfica</title>

    <style>
        body {
            margin: 0;
            height: 100vh;
            display: flex;
            justify-content: center; /* centra horizontal */
            align-items: center;     /* centra vertical */
            background-color: #f0f0f0;
        }

        canvas {
            border: 1px solid black;
        }
    </style>
</head>
<body>

    <canvas id="canvas" width="800" height="600"></canvas>

    <script src="script.js"></script>

</body>
</html>
```

🧠 ¿Qué hace esto?

- `display: flex` → activa el sistema de alineación
- `justify-content: center` → centra horizontalmente
- `align-items: center` → centra verticalmente

- `height: 100vh` → ocupa toda la pantalla

⚠️ Opcional (si quieres que se vea aún más pro)

Si quieres mantener el título, haz esto:

html Copiar código

```html
<body>
    <div style="text-align:center;">
        <h2>Polígono con circunferencias (Rasterización)</h2>
        <canvas id="canvas" width="800" height="600"></canvas>
    </div>
</body>
```

Y en CSS:

css Copiar código

```css
body {
    margin: 0;
    height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}
```

🏁 Resultado

✔️ Canvas centrado en toda la pantalla
✔️ Tu polígono queda perfectamente centrado visualmente
✔️ No rompes nada de tu lógica actual

Si quieres después lo hacemos aún más pro:

- ↪ que el canvas se ajuste a la pantalla 🔥
- o ↪ que el polígono escale automáticamente